

✓ Optic flow task

This notebook illustrates the optic flow task and how to use it with our pretrained fly visual system model and decoder.

Select GPU runtime

To run the notebook on a GPU select Menu -> Runtime -> Change runtime type -> GPU.

Check access to GPU

[Show code](#)

```
-----  
AssertionError                                Traceback (most recent call last)  
/tmp/ipykernel_3399/96976449.py in <cell line: 0>()  
    12  
    13     try:  
--> 14         cuda_name = torch.cuda.get_device_name()  
    15         print(f"Name of the assigned GPU / CUDA device: {cuda_name}")  
    16     except RuntimeError:  
  
----- 2 frames -----  
/usr/local/lib/python3.12/dist-packages/torch/cuda/__init__.py in _lazy_init()  
    415         )  
    416         if not hasattr(torch._C, "_cuda_getDeviceCount"):  
--> 417             raise AssertionError("Torch not compiled with CUDA  
enabled")  
    418         if _cudart is None:  
    419             raise AssertionError(  
  
AssertionError: Torch not compiled with CUDA enabled
```

Next steps: [Explain error](#)

Install Flyvis

The notebook requires installing our package `flyvis`. You may need to restart your session after running the code block below with Menu -> Runtime -> Restart session. Then, imports from `flyvis` should succeed without issue.

Install Flyvis

[Show code](#)

Download pretrained models

Running the code block below will download the pretrained models needed for this notebook.

Download pretrained models

[Show code](#)

✓ The Sintel dataset

We use the Sintel dataset to train our models as described in the paper. More infos about the Sintel dataset can be found on the official Sintel website:

<http://sintel.is.tue.mpg.de/>.

```
import matplotlib.pyplot as plt
import numpy as np
import torch

from flyvis.datasets.sintel import MultiTaskSintel
from flyvis.analysis.animations.sintel import SintelSample
```

The class `MultiTaskSintel` loads, preprocesses, renders, and augments the sintel data. It adheres to the pytorch dataset primitive. It provides the interface to the input data and the output data for the flyvis networks. Note: the fly-eye rendering we use here, we introduce in the notebook on creating custom stimuli already.

This is the full setting:

```
dataset = MultiTaskSintel(
    tasks=["flow"],
    boxfilter=dict(extent=15, kernel_size=13),
    # Because the fly eye rendering is square
    # and sintel is wide, we can crop sintel
    # in width and render three sequences from one.
    # This allows us to statically augment our dataset
    # a bit already before we proceed with the random augmentations.
    # We end up with 3 * 23 sequences.
    vertical_splits=3,
    n_frames=19,
```

```

center_crop_fraction=0.7,
dt=1 / 50,
augment=True,
# From sequences with more than n_frames, we randomly sample the start frame
random_temporal_crop=True,
all_frames=False,
# We resample movie frames to the effective framerate given by 1/dt
resampling=True,
# We interpolate the flow arrows to 1/dt.
interpolate=True,
# We flip with equal probability (using one flip-axis).
p_flip=0.5,
flip_axes=[0, 1],
# We rotate with equal probability (using five fold rotation symmetry of the image)
p_rot=5 / 6,
# We randomly adjust contrast and brightness.
contrast_std=0.2,
brightness_std=0.1,
# We add random white noise pixelwise.
gaussian_white_noise=0.08,
gamma_std=None,
_init_cache=True,
unittest=False,
)

```

```

# The `dataset.arg_df` tracks the sequence index, identity etc.
dataset.arg_df

```

✓ Single sample

First, let's chunk this into smaller digestable pieces.

```

dataset = MultiTaskSintel(
    tasks=["flow"],
    boxfilter=dict(extent=15, kernel_size=13),
    vertical_splits=1,
    dt=1 / 24,
    augment=False,
)

```

The first sample. For the target, the pixel-accurate motion vectors, the color indicates the direction of motion of the respective input pixel. The saturation indicates the magnitude of motion.

```
lum = dataset[0]["lum"]
flow = dataset[0]["flow"]

animation = SintelSample(lum[None], flow[None])
animation.animate_in_notebook()
```

Sintel has more groundtruth annotations. We support depth and flow because we know with some confidence that these are relevant for the fly.

```
dataset = MultiTaskSintel(
    tasks=["depth"],
    boxfilter=dict(extent=15, kernel_size=13),
    vertical_splits=1,
    dt=1 / 24,
    augment=False,
)
```

```
lum1 = dataset[0]["lum"]
depth1 = dataset[0]["depth"]

animation = SintelSample(lum1[None], depth1[None])
animation.animate_in_notebook()
```

✓ Augmenting the dataset step-by-step

We apply rich augmentations to the dataset of naturalistic sequences because the dataset is otherwise relatively small. This might lead to overfitting to, e.g., predicting motion mostly into well-represented directions or of objects of specific contrasts etc. Using rich augmentations, we 'ask' the network to generalize better and invariantly compute motion regardless of direction, contrast, brightness, pixel noise, temporal appearance etc.

✓ Vertical splits

First, we split each sequence into three sequences vertically to leverage a wider extent of the video than if we would only render the center. We precompute these renderings.

```
from flyvis.analysis.visualization.plots import quick_hex_scatter
```

```
dataset = MultiTaskSintel(
    tasks=["flow"],
    boxfilter=dict(extent=15, kernel_size=13),
    vertical_splits=3,
    dt=1 / 24,
    augment=False,
)
```

Sintel has 23 movie sequences originally.

```
len(np.unique(dataset.arg_df.original_index))
```

Each original sequence is 436 pixel in height times 1024 pixel in width in cartesian coordinates.

```
sequence = dataset.cartesian_sequence(0, vertical_splits=1, center_crop_fractic
print(sequence.shape)
```

With the vertical crops, we end up with $3 * 23$ sequences. The `dataset.arg_df` tracks the sequence index, identity etc.

```
dataset.arg_df
```

```
_ = plt.imshow(sequence[0, 0], cmap=plt.cm.binary_r)
```

```
fig, axes = plt.subplots(1, 3)
_ = quick_hex_scatter(dataset[0]['lum'][0].flatten(), fig=fig, ax=axes[0], cbar
_ = quick_hex_scatter(dataset[1]['lum'][0].flatten(), fig=fig, ax=axes[1], cbar
_ = quick_hex_scatter(dataset[2]['lum'][0].flatten(), fig=fig, ax=axes[2], cbar
```

✓ Random temporal crops

We train on 19 frames ~ 792ms movie. Most sequences have 49 frames. To use the whole temporal content, we stochastically sample start and end frame ~ ((1, 19), (2, 20), ..., (31, 49)).

```
dataset = MultiTaskSintel(
    tasks=["flow"],
    boxfilter=dict(extent=15, kernel_size=13),
    vertical_splits=3,
    n_frames=19,
```

```

dt=1 / 24,
augment=True,
random_temporal_crop=True,
all_frames=False,
resampling=False,
interpolate=False,
p_flip=0,
p_rot=0,
contrast_std=None,
brightness_std=None,
gaussian_white_noise=None,
)

```

```

# These two samples from the same original sequence should have stochastically
lum1 = dataset[0]['lum']
lum2 = dataset[0]['lum']

```

```

animation = SintelSample(lum1[None], lum2[None], title2="input")
animation.animate_in_notebook()

```

✓ Flips and rotations

Next, we flip stochastically across 2 axes and or rotate a random number of times around the center. We implement this to be fast to do so at runtime.

```

dataset = MultiTaskSintel(
    tasks=["flow"],
    boxfilter=dict(extent=15, kernel_size=13),
    vertical_splits=3,
    n_frames=19,
    dt=1 / 24,
    augment=True,
    random_temporal_crop=False,
    all_frames=False,
    resampling=False,
    interpolate=False,
    p_flip=1 / 2,
    p_rot=5 / 6,
    contrast_std=None,
    brightness_std=None,
    gaussian_white_noise=None,
)

```

```

# These two samples from the same original sequence should have stochastically
lum1 = dataset[0]['lum']

```

```
lum2 = dataset[0]['lum']
```

```
animation = SintelSample(lum1[None], lum2[None], title2="input")  
animation.animate_in_notebook()
```

Flow vectors need to be flipped and rotated accordingly.

```
# These two samples from the same original sequence should have stochastically  
data = dataset[0]  
lum1 = data['lum']  
flow1 = data['flow']
```

```
animation = SintelSample(lum1[None], flow1[None])  
animation.animate_in_notebook()
```

✓ Further augmentations

Besides that, we also augment the input with random contrasts and brightnesses and random gaussian pixel noise, while the motion stays the same. This pretends that the same motion takes place under different illumination conditions and signal to noise ratios.

```
dataset = MultiTaskSintel(  
    tasks=["flow"],  
    boxfilter=dict(extent=15, kernel_size=13),  
    vertical_splits=3,  
    n_frames=19,  
    dt=1 / 24,  
    augment=True,  
    random_temporal_crop=False,  
    all_frames=False,  
    resampling=False,  
    interpolate=False,  
    p_flip=0,  
    p_rot=0,  
    contrast_std=0.2,  
    brightness_std=0.1,  
    gaussian_white_noise=0.08,  
)
```

```
# These two samples from the same original sequence have  
# stochastically different contrast, brightness and pixel-wise noise.
```

```
lum1 = dataset[0]['lum']  
lum2 = dataset[0]['lum']
```

```
animation = SintelSample(lum1[None], lum2[None], title2="input")  
animation.animate_in_notebook()
```

✓ Framerate of the dataset and integration time step

The Sintel dataset is originally rendered at 24 frames per second, i.e., one frame every 42ms. The fruit fly neurons are able to respond to temporal differences as fast as 5-20ms. Therefore, we resample every frame multiple times to pretend that the movie was originally sampled at such a faster framerate. For the motion fields, we interpolate flow vectors in time instead of resampling them, which hopefully gives a better learning signal to the network. We have to trade-off speed of the numerical integration and memory consumption during optimization with the simulation accuracy by choosing time steps between 5-20ms. We chose to train networks at the upper bound of 20ms and evaluate them more accurately at 5-10ms.

```
dataset = MultiTaskSintel(  
    tasks=["flow"],  
    boxfilter=dict(extent=15, kernel_size=13),  
    vertical_splits=3,  
    n_frames=19,  
    dt=1 / 50,  
    augment=False,  
    resampling=True,  
    interpolate=True,  
)
```

```
# Now, every input frame appears twice and target frames are interpolated.  
data = dataset[0]  
lum1 = data['lum']  
flow1 = data['flow']
```

```
animation = SintelSample(lum1[None], flow1[None])  
animation.animate_in_notebook()
```

✓ Computing responses to the Sintel data

Before we get to training a network, we look at a few responses to these type of sequences of individual neurons.

```
from flyvis.network import NetworkView, Network
from flyvis.utils.activity_utils import LayerActivity

from flyvis.datasets.sintel import MultiTaskSintel
```

```
# new network instance
network = Network()

# Alternative: uncomment to use a pretrained network
# network_view = NetworkView(results_dir / "flow/0000/000")
# network = network_view.init_network(network)
```

```
layer_activity = LayerActivity(None, network.connectome, keepref=True)
```

```
dataset = MultiTaskSintel(
    tasks=["flow"],
    boxfilter=dict(extent=15, kernel_size=13),
    vertical_splits=1,
    n_frames=19,
    dt=1 / 50,
    augment=False,
    resampling=True,
    interpolate=True,
)
```

```
stationary_state = network.fade_in_state(1.0, dataset.dt, dataset[0]["lum"][[0]]
responses = network.simulate(
    dataset[0]["lum"][None], dataset.dt, initial_state=stationary_state
).cpu()
```

```
plt.figure(figsize=[3, 2])
layer_activity.update(responses)
r = layer_activity.central.T4c.squeeze().numpy()
time = np.arange(0, r.shape[0], 1) * dataset.dt
plt.plot(time, r)
plt.xlabel("time in s")
plt.ylabel("voltage (a.u.)")
plt.title("response of central T4c cell")
```

✓ Decoding the task from neural activity

We need to predict the pixel-accurate flow field that Sintel gives us. For that we decode the voltages of a bunch of cell types. The decoder and the network are trained end-to-end. Here an example of a forward pass through the whole pipeline in code.

```
from flyvis.datasets.sintel import MultiTaskSintel
from flyvis.task.decoder import DecoderGAVP
```

```
network = Network()
```

```
decoder = DecoderGAVP(network.connectome, shape=[8, 2], kernel_size=5)
```

```
dataset = MultiTaskSintel(
    tasks=["flow"],
    boxfilter=dict(extent=15, kernel_size=13),
    vertical_splits=1,
    all_frames=True,
    dt=1 / 50,
    augment=False,
    resampling=True,
    interpolate=True,
)
```

```
data = dataset[0]
lum = data["lum"]
flow = data["flow"]

stationary_state = network.fade_in_state(1.0, dataset.dt, lum[[0]])
responses = network.simulate(lum[None], dataset.dt, initial_state=stationary_st
```

```
y_pred = decoder(responses)
```

We predict motion with an untrained decoder from an untrained network with randomly initialized parameters. We do not expect this to work.

```
animation = SintelSample(lum[None], flow[None], prediction=y_pred.detach().cpu(
    animation.animate_in_notebook(frames=np.arange(lum.shape[0]))[:10]))
```

```
((y_pred - flow) ** 2).sqrt().mean()
```

✓ Training network and decoder on a single batch

We now train the network on a single batch to validate that the pipeline works. We do not expect these networks to generalize their function.

```
from tqdm.notebook import tqdm
from torch.optim import Adam
from torch.utils.data import DataLoader

from flyvis.network import Network
from flyvis.task.decoder import DecoderGAVP

from flyvis.datasets.sintel import MultiTaskSintel
from flyvis.task.objectives import l2norm, epe
```

```
network = Network()
```

```
decoder = DecoderGAVP(network.connectome, shape=[8, 2], kernel_size=5)
```

```
dataset = MultiTaskSintel(
    tasks=["flow"],
    boxfilter=dict(extent=15, kernel_size=13),
    vertical_splits=1,
    n_frames=19,
    dt=1 / 50,
    augment=False,
    resampling=True,
    interpolate=True,
)
```

```
t_pre = 0.5
dt = 1 / 50
batch_size = 4
train_loader = DataLoader(dataset, batch_size=batch_size)
```

```
optimizer = Adam((*network.parameters(), *decoder.parameters()), lr=1e-5)
```

```
batch = next(iter(train_loader))
```

```
loss_fn = epe
```

```
epochs = 1000

errors = []

initial_state = network.steady_state(t_pre, dt, batch_size)
```

```

for e in tqdm(range(epochs)):
    lum = batch["lum"]
    flow = batch["flow"]

    optimizer.zero_grad()
    network.stimulus.zero()
    network.stimulus.add_input(lum)

    activity = network(network.stimulus(), dt=1 / 50, state=initial_state)
    y_pred = decoder(activity)

    batch_error = loss_fn(y_pred, flow)
    errors.append(batch_error.cpu().item())
    batch_error.backward()
    optimizer.step()

    if e % 10 == 0:
        print(f"Epoch {e}: {batch_error.item()}")

```

```
plt.plot(errors)
```

We expect that the prediction from this overfitted network on the sample it was trained on is ok.

```

data = dataset[0]
lum = data["lum"]
flow = data["flow"]

stationary_state = network.fade_in_state(1.0, dataset.dt, lum[[0]])
responses = network.simulate(lum[None], dataset.dt, initial_state=stationary_st

```

```
y_pred = decoder(responses)
```

```

animation = SintelSample(lum[None], flow[None], prediction=y_pred.detach().cpu()
animation.animate_in_notebook()

```

```
((y_pred - flow) ** 2).sqrt().mean()
```

✓ Evaluating trained networks

```

from flyvis import results_dir
from flyvis.network import NetworkView
from flyvis.utils.activity_utils import LayerActivity

```

```
from flyvis.datasets.sintel import MultiTaskSintel
from flyvis.task.decoder import DecoderGAVP
```

```
# we load the best task-performing model from the presorted ensemble
network_view = NetworkView(results_dir / "flow/0000/000")
```

```
network = network_view.init_network()
```

```
network_view.dir.config.task.decoder
```

```
decoder = network_view.init_decoder()["flow"]
```

```
dataset = MultiTaskSintel(
    tasks=["flow"],
    boxfilter=dict(extent=15, kernel_size=13),
    vertical_splits=1,
    all_frames=False,
    n_frames=19,
    dt=1 / 50,
    augment=False,
    resampling=True,
    interpolate=True,
)
```

```
data = [dataset[i] for i in range(4)]
lum = torch.stack([d["lum"] for d in data])
flow = torch.stack([d["flow"] for d in data])

stationary_state = network.fade_in_state(1.0, dataset.dt, lum[:, 0])
responses = network.simulate(lum, dataset.dt, initial_state=stationary_state)
```

```
y_pred = decoder(responses)
```

We expect this network to generalize across sequences. This network sees motion into all directions.

```
animation = SintelSample(lum, flow, prediction=y_pred.detach().cpu())
animation.animate_in_notebook()
```

We expect the accuracy is not as good as the overfitted example because this network generalized across the whole-dataset.

```
((y_pred - flow) ** 2).sqrt().mean()
```

✓ Evaluating ensembles

Last, we evaluated the task error of the 50 trained networks on a held out set of sequences. We evaluated the task error across all checkpoints during training and show the minimal one in the histogram below. This checkpoint we analyse with respect to it's tuning predictions as shown in the next notebooks.

```
from flyvis import EnsembleView
```

```
EnsembleView(  
    "flow/0000",  
    best_checkpoint_fn_kwargs={"validation_subdir": "validation", "loss_file_name":  
    })
```

```
ensemble.task_error_histogram()
```